

Tests For CAInventoryAPI_Implementation

Method	Test ID	Parameter Values	Comment	Expected Outcome
getInventory(productId : string) : string	1	productId = "P10"	Product exists within the inventory	"Success" Not null returned string whilst containing product entries
getInventory(productId : string) : string	2	productId = "Unknown"	Product does not exist	"Failure" Empty string returned or an error message
getInventory(productId : string) : string	3	productId = " "	ID is invalid	"Failure" A failure message is returned (string)
withdrawInventory (item : string, amount : int) : boolean	4	Item = "P10" Amount = 1	Product exists with available stock	"Success" Boolean returns true, quantity is lessened by 1
withdrawInventory (item : string, amount : int) : boolean	5	Item = "P10" Amount = 2	Product exists with unavailable stock (Insufficient)	"Failure" Boolean returns false, no changes are made to the inventory
withdrawInventory (item : string, amount : int) : boolean	6	Item = "Unknown" Amount = 1	Product ID is not found within the inventory	"Failure" Boolean returns false, no changes are made to the inventory
withdrawInventory (item : string, amount : int) : boolean	7	Item = "P10" Amount = 0	No withdrawal from inventory	"Failure" Boolean returns false, no changes are made to the inventory
withdrawInventory (item : string, amount : int) : boolean	8	Item = "P10" Amount = -1	Negative value is invalid	"Failure" Boolean returns false, no changes are made to the inventory

CAInventoryAPI()	9	(none)	Normal creation	"Success" Object is created successfully and the structure of the inventory is correctly initialised
CAInventoryAPI()	10	(none)	More than one created	"Success" Two different objects should not share data and should stay independent
createNewOrder(orderId : string, orderDetails : string)	11	orderId = "ORD-01" orderDetails = "P10:1, P20:2"	The order is valid	"Success" The order is created and recorded and then linked to inventory
createNewOrder(orderId : string, orderDetails : string)	12	orderId = " " orderDetails = "P10:1, P20:2"	The order is missing an order ID	"Failure" The order is not created
createNewOrder(orderId : string, orderDetails : string)	13	orderId = "ORD-01" orderDetails = " "	The order is missing order details	"Failure" The order is not created
createNewOrder(orderId : string, orderDetails : string)	14	orderId = "ORD-01" orderDetails = "Unknown"	The product does not exist	"Failure" The order is either not created or produces an error message

Tests For ICAInventoryAPI

Method	Test ID	Parameter Values	Comment	Expected Outcome
--------	---------	------------------	---------	------------------

withdrawInventory (item : string, amount : int) : boolean	I-01	Item = "P10" Amount = 1	Product exists with available stock	"Success" Boolean returns true, quantity is lessened by 1
withdrawInventory (item : string, amount : int) : boolean	I-02	Item = "P10" Amount = 2	Product exists with unavailable stock (Insufficient)	"Failure" Boolean returns false, no changes are made to the inventory
withdrawInventory (item : string, amount : int) : boolean	I-03	Item = "Unknown" Amount = 1	Product ID is not found within the inventory	"Failure" Boolean returns false, no changes are made to the inventory
withdrawInventory (item : string, amount : int) : boolean	I-04	Item = "P10" Amount = 0	No withdrawal from inventory	"Failure" Boolean returns false, no changes are made to the inventory
withdrawInventory (item : string, amount : int) : boolean	I-05	Item = "P10" Amount = -1	Negative value is invalid	"Failure" Boolean returns false, no changes are made to the inventory
getInventory(productId : string) : string	I-06	ProductId = "P10"	A product exists	"Success" Returns string of product details
getInventory(productId : string) : string	I-07	ProductId = "Unknown"	A product does not exist	"Failure" Returns string of an error/unsuccessful message
getInventory(productId : string) : string	I-08	ProductId = " "	ID is invalid	"Failure" Nothing happens because the ID is invalid
createNewOrder(orderId : string, orderDetails : string)	I-09	orderId = "ORD-01"orderDetail	The order is valid	"Success" The order is created and

		s = "P10:1, P20:2"		recorded and then linked to inventory
createNewOrder(orderId : string, orderDetails : string)	I-10	orderId = " " orderDetails = "P10:1, P20:2"	The order is missing an order ID	"Failure" The order is not created
createNewOrder(orderId : string, orderDetails : string)	I-11	orderId = "ORD-01" orderDetails = " "	The order is missing order details	"Failure" The order is not created
createNewOrder(orderId : string, orderDetails : string)	I-12	orderId = "ORD-01" orderDetails = "Unknown"	The product does not exist	"Failure" The order is either not created or produces an error message

Tests For IPUPaymentAPI

Method	Test ID	Parameter Values	Comment	Expected Outcome
recordPayment(cardNumber : int, name : string, expiryDate : string, cardType : string, amount : int) : boolean	P-01	cardNumber = 10000000 name = "A Smith" expiryDate = "26/28" cardType = "Visa" Amount = 100	Payment details that are valid	"Success" Payment gets recorded and then the paymentID gets stored

recordPayment(cardNumber : int, name : string, expiryDate : string, cardType : string, amount : int) : boolean	P-02	cardNumber = 10000000 name = "A Smith" expiryDate = "26/25" cardType = "Visa" Amount = 100	Card has expired	"Failure" Payment is invalid because the card has expired and therefore does not go through/get recorded
recordPayment(cardNumber : int, name : string, expiryDate : string, cardType : string, amount : int) : boolean	P-03	cardNumber = 10 name = "A smith" expiryDate = "26/02" cardType = "Visa" Amount = 100	Invalid card number	"Failure" Payment is invalid because the card has an invalid card number and therefore does not go through/get recorded
recordPayment(cardNumber : int, name : string, expiryDate : string, cardType : string, amount : int) : boolean	P-04	cardNumber = 10000000 name = " " expiryDate = "26/02" cardType = "Visa" Amount = 100	There is no name for the cardholder	"Failure" Payment is invalid because the card has no name for the cardholder and therefore does not go through/get recorded
recordPayment(cardNumber : int, name : string, expiryDate : string, cardType : string, amount : int) : boolean	P-05	cardNumber = 10000000 name = "A Smith"	Does not take any amount out of the card	"Failure" Payment is invalid because there is no amount being taken out of the

		expiryDate = "26/02" cardType = "Visa" Amount = 0		card and therefore does not go through/get recordedt being taken out of the card
refundPayment(paymentId : int) : boolean	R- 01	paymentId = 1001	Valid payment, refundabl e	"Success" The status of the payment becomes refunded
refundPayment(paymentId : int) : boolean	R- 02	paymentId = 1001	Refund has been processed already before	"Failure" The status of the payment has already been declared as refunded
refundPayment(paymentId : int) : boolean	R- 03	paymentId = 10000	The payment is not found	"Failure" The refund does not go through because the payment is not found
refundPayment(paymentId : int) : boolean	R- 04	paymentId = -1	ID is invalid	"Failure" The refund does not go through because the ID is invalid
getPaymentStatus(paymentId : string) : string	S- 01	paymentId = "1001"	Payment that already exists	"Success" Returns string with a successful paid message
getPaymentStatus(paymentId : string) : string	S- 02	paymentId = "1001"	Post refund	"Success" Returns string with a refunded message
getPaymentStatus(paymentId : string) : string	S- 03	paymentId = "10000"	Payment status is not found	"Failure" Returns string of an error/unsuccessfu l message

getPaymentStatus(paymentId : string) : string	S-04	paymentId = " "	ID is invalid	"Failure" Nothing happens because the ID is invalid
---	------	-----------------	---------------	--

Tests for ISALoginAPI

Method	Test ID	Parameter Values	Comment	Expected Outcome
login(username : string, password : string) : boolean	L-01	username = "User1" password = "Pass1"	Credentials are valid	"Success" Boolean returns true and a user session is created
login(username : string, password : string) : boolean	L-02	username = "User1" password = "Pass2"	Password is invalid	"Failure" Boolean returns false and a user session is not created/does not start because of the invalid password
login(username : string, password : string) : boolean	L-03	username = " " password = "Pass1"	Username is missing	"Failure" Boolean returns false and a user session is not created/does not start because of the missing username
login(username : string, password : string) : boolean	L-04	username = "User1" password = " "	Password is missing	"Failure" Boolean returns false and a user session is created/does not start because of the missing password
isAuthenticated(merchantId : string) :boolean	A-01	merchantId = "M1"	The merchant has logged in successfully	"Success" Boolean returns true and the merchant logs in

				with the user session starting
isAuthenticated(merchantId : string) :boolean	A-02	merchantId = "M1"	The merchant either logged out or the session expired	"Failure" Boolean returns false and the merchant either logs out or the user session expired
isAuthenticated(merchantId : string) :boolean	A-03	merchantId = "Unknown"	The merchant does not exist	"Failure" Boolean returns false and nothing happens because the merchant does not exist
isAuthenticated(merchantId : string) :boolean	A-04	merchantId = " "	The merchant ID is invalid	"Failure" Boolean returns false and nothing happens because the merchant ID is invalid
logout(userId : string) : void	O-01	userId = "U1"	A logged in user logs out	"Success" Session ends and then any future activity will end in the boolean returning false
logout(userId : string) : void	O-02	userId = "U1"	A logged in user logs out twice so it is called again	"Success" The user stays logged out and nothing happens
logout(userId : string) : void	O-03	userId = " "	The user ID is invalid	"Failure" Nothing happens because the ID is invalid

Tests for ISAOrderAPI

Method	Test ID	Parameter Values	Comment	Expected Outcome
PlaceOrder(merchantId : int, orderDetails : string) :string	B-01	merchantId = 101, orderDetails = "Paracetamol x2"	Valid merchant ID and available stock items.	"Success": Method returns order confirmation
PlaceOrder(merchantId : int, orderDetails : string) :string	B-02	merchantId = 999, orderDetails = "Aspirin x1"	Invalid/Non-existent merchant ID.	"Failure": The method returns error message "Invalid Merchant ID"; and no order recorded.
PlaceOrder(merchantId : int, orderDetails : string) :string	B-03	merchantId = 101, orderDetails = "Insulin x500"	Order quantity exceeds available stockAvailability.	"Failure": Method returns "Insufficient Stock"; order is rejected.
PlaceOrder(merchantId : int, orderDetails : string) :string	B-04	merchantId = 101, orderDetails = ""	orderDetails string is empty.	"Failure": Method returns "Insufficient Stock"; order is rejected.
getOrder(orderID : string): string	H-01	orderID = "O5001"	Valid Order ID that exists in the database.	"Success": Returns products, quantities,

				and total price.
getOrder(orderID : string): string	H-02	orderID = “-5”	Order ID that does not exist in the database.	"Failure": Returns "Order Not Found" error.
getInvoice(orderID : string): string	H-01	orderID = “O5002 ‘	Valid Order ID for an existing, completed sale	Success The method should generate the invoice for the order
getInvoice(orderID : string): string	J-02	orderID = “-5 ‘	Attempting to retrieve an invoice for an ID that does not exist.	"Failure": Returns an error string: "Order ID not found."
getInvoice(orderID : string): string	J-03	orderID = ““	Attempting to retrieve an invoice for an empty input	"Failure": Returns an error string: "Please input."
getOrderStatus(orderId : string) : string	K-01	orderID = “P1001 ‘	Valid Order ID for an existing active transaction.	"Success": Returns a string indicating current status (e.g., "Dispatched").
getOrderStatus(orderId : string) : string	K-02	orderID = “-5 ‘	Attempting to check status for a non-existent Order ID.	"Failure": Returns error string: "Order ID not found."
getOrderStatus(orderId : string) : string	K-03	orderID = ““	Attempting to check status	"Failure": Returns

			for an empty input	error string: "Please input."
updateDispatchInfo(orderId : string, dispatchInfo : string) : boolean	Q-01	orderId = "P1001", dispatchInfo="Dispatched"	Valid Order ID with a complete dispatch update string.	"Success" Method should successfully update the info to the ID
updateDispatchInfo(orderId : string, dispatchInfo : string) : boolean	Q-02	orderId = "-5", dispatchInfo="Dispatched"	Invalid Order ID with a complete dispatch update string.	"Failure": Returns error string: "Order ID not found."
updateDispatchInfo(orderId : string, dispatchInfo : string) : boolean	Q-03	orderId = "P1001", dispatchInfo=""	Valid Order ID with an empty dispatch update string.	"Failure": Returns error string: "Please input info."
updateDispatchInfo(orderId : string, dispatchInfo : string) : boolean	Q-04	orderId = "", dispatchInfo=""	Empty Order ID with an empty dispatch update string.	"Failure": Returns error string: "Please input OrderID."
logout(userId : string) : void	C-02	userId = "U1"	A logged in user logs out twice so it is called again	"Success" The user stays logged out and nothing happens
logout(userId : string) : void	C-03	userId = ""	The user ID is invalid	"Failure" Nothing happens because the ID is invalid

Tests for IPUCommsAPI

Method	Test ID	Parameter Values	Comment	Expected Outcome
sendEmail(subject : string, body : string, email : string):boolean	E-01	subject = "Debt Reminder", body = "Your balance is...", email = "customer@mail.com"	Valid input with all required fields filled correctly.	"Success": Boolean returns true. The email is successfully passed to the PU for delivery.
sendEmail(subject : string, body : string, email : string):boolean	E-02	subject = "Reminder", body = "Payment due", email = "customer@@@gmail.com"	Attempting to send with an incorrectly formatted email address.	"Failure": Boolean returns false. System prevents the call due to invalid email structure.
sendEmail(subject : string, body : string, email : string):boolean	E-03	subject = "", body = "", email = "customer@mail.com"	Sending an email with empty	"Failure": Boolean returns false

email : string):boolean			subject or body strings.	as method requires content in subject and body
----------------------------	--	--	--------------------------------	--

Tests for ISACatalogueAPI

Method	Test ID	Parameter Values	Comment	Expected Outcome
searchProduct(keyword : string) : string	D-01	keyword = "Paracetamol"	Searching for a valid product name that exists in the SA database.	Returns a string containing product ID, description, and current price.
searchProduct(keyword : string) : string	D-02	keyword = "ABCD"	Searching for a product that does not exist in the catalogue.	"Failure": Returns "Product not found" or an empty result string.
searchProduct(keyword : string) : string	D-03	keyword = ""	Testing the boundary case with an empty search string.	"Failure": Returns "Invalid Input" or triggers a

				full list view if intended
getProductById(productId : int) : string	X-01	productId = "P1001"	Valid ID for an existing product (e.g., Paracetamol)	Returns a string with full details
getProductById(productId : int) : string	X-02	productId = "P9999"	ID that does not exist in the SA database.	"Failure": Returns "Product ID Not Found" error string.
addProduct(productId : string, name:string, price : double, stock : int) : boolean	Y-01	productId="P1002" name="Cough Syrup" price=10.50 stock = 5000	Adding a new, valid product with all fields correctly filled.	Product is added to the SA database.
addProduct(productId : string, name:string, price : double, stock : int) : boolean	Y-02	productId="" name="" price="" stock = ""	Any data is missing	Failure Data is missing
addProduct(productId : string, name:string, price : double, stock : int) : boolean	Y-03	productId="P1002" name="Cough Syrup" price=-10.50 stock = 5000	Adding a new, valid product with all fields correctly filled, but price is negative	Failure Price can not be negative. Catch the error
addProduct(productId : string, name:string, price : double, stock : int) : boolean	Y-03	productId="P1002" name="Cough Syrup" price=10.50 stock = 5000	Adding a new, valid product with all fields correctly filled, but ID repeats	Failure Item already exists with this ID
updateStock(productId : string, quantityToAdd : int) : void	Z-01	productId = "P1001" quantityToAdd = 200	Increasing stock by 200	Method successfully increases the stock by

				the given amount
updateStock(productId : string, quantityToAdd : int) : void	Z-02	productId = "-5" quantityToAdd = 200	Attempting an ID which doesn't exist	Failure Error caught No such ID exists
deleteProduct(productId : string) : void	T-01	productId = "P1001"	Attempting an ID which exists	Success Product is deleted and all its data
deleteProduct(productId : string) : void	T-02	productId = "-5"	Attempting an ID which doesn't exist	Failure No such ID exists in the database
updateProduct(productId : string, name : string, price : double) : void	M-01	productId = "P1001" name = "Paracetamol" price = 3.50	Updating all fields for an existing product ID.	Success The product should be updated
updateProduct(productId : string, name : string, price : double) : void	M-02	productId = "-5" name = "Paracetamol" price = 3.50	Updating all fields for an existing product ID, but ID is non-existent	Failure Catch error No such ID exists in the database
updateProduct(productId : string, name : string, price : double) : void	M-03	productId = "P1001" name = "Paracetamol" price = -3.50	Updating all fields for an existing product ID, but price is negative	Failure Catch error Price has to be a positive number